

MEMORIA PRÁCTICA ADICIONAL PROGRAMACIÓN EVOLUTIVA



ALGORITMO NEAT PARA FLAPPY BIRD

CURSO 2024-2025

AUTOR
GUILLERMO SÁNCHEZ PÉREZ

FACULTAD DE INFORMÁTICA - UNIVERSIDAD COMPLUTENSE DE MADRID

Contenido

1. Introducción.....	3
2. Estructura del Proyecto	3
2.1. Paquete com.neat.flappybirdneat	3
2.2. Paquete com.neat.flappybirdneat.game	3
2.3. Paquete com.neat.flappybirdneat.neat.....	3
2.4. Paquete com.neat.flappybirdneat.neural	3
2.5. Paquete com.neat.flappybirdneat.view	4
3. Implementación del Algoritmo NEAT	4
3.1. Representación de los Agentes.....	4
3.2. Proceso Evolutivo.....	4
3.3. Lógica del Juego.....	5
4. Interfaz Gráfica.....	5
4.1. Visualización de la Simulación	5
4.2. Controles de Simulación	5
4.3. Herramientas de Análisis.....	5
5. Detalles de Implementación.....	6
5.1. Red Neuronal Feedforward.....	6
5.2. Operadores Genéticos.....	7
5.3. Toma de Decisiones de los Agentes	8
5.4. Bucle de Juego.....	9
6. Características Adicionales	10
6.1. Control de Velocidad de Simulación.....	10
6.2. Visualización de la Red Neuronal.....	11
6.3. Gráficos de Rendimiento	11
6.4. Controles Avanzados	11
7. Conclusiones	11

1. Introducción

Este proyecto implementa una versión del algoritmo NEAT (NeuroEvolution of Augmenting Topologies) aplicado al famoso juego Flappy Bird. El objetivo es desarrollar un sistema capaz de entrenar agentes controlados por redes neuronales que, a través de un proceso evolutivo, aprendan a jugar al juego de forma autónoma y eficiente.

El sistema está desarrollado en Java y utiliza JavaFX para la interfaz gráfica. La implementación permite visualizar el proceso de aprendizaje en tiempo real, ajustar parámetros del algoritmo y analizar el rendimiento de las diferentes generaciones de agentes.

2. Estructura del Proyecto

El proyecto está organizado en varios paquetes, cada uno responsable de una función específica:

2.1. Paquete `com.neat.flappybirdneat`

- **FlappyBirdNEAT**: Clase principal que integra todo el sistema y maneja la interfaz gráfica.

2.2. Paquete `com.neat.flappybirdneat.game`

- **FlappyBirdGame**: Implementa la lógica del juego, gestionando los tubos, colisiones y puntuación.
- **Pipe**: Representa los obstáculos (tubos) en el juego, con su posición, tamaño y comportamiento.

2.3. Paquete `com.neat.flappybirdneat.neat`

- **FlappyBirdAgent**: Representa un agente controlado por una red neuronal.
- **Population**: Gestiona una población de agentes, aplicando selección natural, cruce y mutación.

2.4. Paquete `com.neat.flappybirdneat.neural`

- **NeuralNetwork**: Implementa una red neuronal feedforward con una capa oculta, incluyendo funcionalidades para mutación y cruce genético.

2.5. Paquete com.neat.flappybirdneat.view

- **NeuralNetworkWindow:** Ventana que visualiza la red neuronal del mejor agente.
- **FitnessGraphWindow:** Ventana que muestra gráficos de la evolución del fitness a lo largo de las generaciones.

3. Implementación del Algoritmo NEAT

3.1. Representación de los Agentes

Cada agente está representado por la clase FlappyBirdAgent, que contiene:

1. **Red Neuronal:** Una instancia de NeuralNetwork que decide cuándo el pájaro debe saltar.
2. **Atributos Físicos:** Posición, velocidad y estado (vivo o muerto).
3. **Fitness:** Valor que representa qué tan bien ha jugado el agente.

La red neuronal de cada agente tiene:

- **4 Entradas:** Posición Y del pájaro, velocidad, distancia al próximo tubo y altura del hueco.
- **8 Neuronas en la capa oculta.**
- **1 Salida:** Determina si el pájaro debe saltar (>0.5) o no.

3.2. Proceso Evolutivo

El algoritmo evolutivo se implementa en la clase Population y sigue los siguientes pasos:

1. **Inicialización:** Se crea una población inicial de agentes con redes neuronales inicializadas aleatoriamente.
2. **Evaluación:** Cada agente juega al Flappy Bird hasta que muere, acumulando fitness (aumenta con cada frame que sobrevive).
3. **Selección:** Se seleccionan padres para la siguiente generación mediante un método de selección por ruleta (probabilidad proporcional al fitness).
4. **Elitismo:** El mejor agente de la generación actual pasa directamente a la siguiente.
5. **Cruce:** Se cruzan dos padres para crear un hijo, combinando los pesos de sus redes neuronales.

```
java
child.getBrain().setBrain(NeuralNetwork.crossover(parent1.getBrain(), parent2.getBrain()));
```

6. **Mutación:** Se aplican mutaciones aleatorias a los pesos y sesgos de la red neuronal del hijo.

```
java  
child.getBrain().mutate(mutationRate);
```

7. **Reemplazo:** La nueva generación reemplaza a la anterior y el ciclo continúa.

3.3. Lógica del Juego

La clase FlappyBirdGame implementa la lógica del juego:

1. **Generación de Tubos:** Se crean tubos a intervalos regulares.
2. **Detección de Colisiones:** Se detectan colisiones entre los pájaros y los tubos/bordes.
3. **Puntuación:** Se incrementa cuando un pájaro pasa por un tubo.

La clase Pipe representa los obstáculos:

- Posición y tamaño del tubo.
- Posición y tamaño del hueco.
- Método para detectar colisiones.

4. Interfaz Gráfica

La interfaz gráfica, implementada con JavaFX, permite:

4.1. Visualización de la Simulación

- Canvas principal que muestra el juego en tiempo real.
- Opción para mostrar todos los agentes o solo el mejor.
- Indicadores de generación, agentes vivos, puntuación y mejor fitness.

4.2. Controles de Simulación

- Ajuste de parámetros: tamaño de población y número máximo de generaciones.
- Control de velocidad de simulación.
- Botones para pausar/continuar, saltar a la siguiente generación y reiniciar.

4.3. Herramientas de Análisis

- Visualización de la red neuronal del mejor agente en tiempo real.
- Gráfico de evolución del fitness a lo largo de las generaciones.

5. Detalles de Implementación

5.1. Red Neuronal Feedforward

Una red neuronal feedforward es un tipo de modelo computacional que procesa información en una sola dirección, desde la entrada hacia la salida, sin conexiones cíclicas. La clase NeuralNetwork implementa una red neuronal feedforward con una capa oculta:

```
java
public double[] feedForward(double[] inputs) {
    // Activación de la capa oculta
    double[] hiddenLayer = new double[hiddenSize];
    for (int i = 0; i < hiddenSize; i++) {
        double sum = biasHidden[i];
        for (int j = 0; j < inputSize; j++) {
            sum += inputs[j] * weightsInputHidden[j][i];
        }
        hiddenLayer[i] = sigmoid(sum);
    }

    // Activación de la capa de salida
    double[] outputs = new double[outputSize];
    for (int i = 0; i < outputSize; i++) {
        double sum = biasOutput[i];
        for (int j = 0; j < hiddenSize; j++) {
            sum += hiddenLayer[j] * weightsHiddenOutput[j][i];
        }
        outputs[i] = sigmoid(sum);
    }

    return outputs;
}
```

El proceso comprende:

1. **Suma Ponderada:** Cada neurona recibe valores de entrada multiplicados por pesos y sumados con un sesgo (bias).
2. **Función de Activación:** La función sigmoid transforma esta suma en un valor entre 0 y 1, simulando la activación neuronal.

```
java
private double sigmoid(double x) {
    return 1.0 / (1.0 + Math.exp(-x));
}
```

5.2. Operadores Genéticos

Cruce (Crossover)

El operador de cruce combina dos redes neuronales padres para crear un hijo. Para cada peso y sesgo, se selecciona aleatoriamente el valor de uno de los padres:

```
java
public static NeuralNetwork crossover(NeuralNetwork parent1,
NeuralNetwork parent2) {
    NeuralNetwork child = new NeuralNetwork(
        parent1.inputSize,
        parent1.hiddenSize,
        parent1.outputSize
    );

    Random random = new Random();

    // Cruzar pesos de entrada a capa oculta
    for (int i = 0; i < parent1.inputSize; i++) {
        for (int j = 0; j < parent1.hiddenSize; j++) {
            if (random.nextBoolean()) {
                child.weightsInputHidden[i][j] =
parent1.weightsInputHidden[i][j];
            } else {
                child.weightsInputHidden[i][j] =
parent2.weightsInputHidden[i][j];
            }
        }
    }
}
```

```

        }
    }
}

// (Similar para el resto de pesos y sesgos)

return child;
}

```

Mutación

El operador de mutación aplica pequeños cambios aleatorios a los pesos y sesgos de la red neuronal, introduciendo variabilidad genética:

```

java
public void mutate(double mutationRate) {
    // Mutar pesos de capa de entrada a capa oculta
    for (int i = 0; i < inputSize; i++) {
        for (int j = 0; j < hiddenSize; j++) {
            if (random.nextDouble() < mutationRate) {
                weightsInputHidden[i][j] += random.nextGaussian() *
0.1;
            }
        }
    }

    // (Similar para el resto de pesos y sesgos)
}

```

El parámetro `mutationRate` determina la probabilidad de que cada peso cambie, mientras que `random.nextGaussian() * 0.1` aplica un cambio aleatorio con distribución normal centrada en 0.

5.3. Toma de Decisiones de los Agentes

El método `think` de `FlappyBirdAgent` toma las entradas del entorno, las normaliza y las pasa a la red neuronal:

```

java

```



```

public void think(float birdY, float distanceToNextPipe, float
heightOfNextPipe, float nextPipeGapSize) {

    // Normalizar entradas

    double[] inputs = new double[4];

    inputs[0] = birdY / 600.0;           // Posición Y
    normalizada

    inputs[1] = velocity / 15.0;        // Velocidad
    normalizada

    inputs[2] = distanceToNextPipe / 800.0; // Distancia
    normalizada

    inputs[3] = heightOfNextPipe / 600.0; // Altura del hueco
    normalizada

    double[] outputs = brain.feedForward(inputs);

    // Si la salida es mayor que 0.5, el pájaro salta
    if (outputs[0] > 0.5) {
        jump();
    }
}

```

La normalización es crucial porque lleva todos los valores de entrada a un rango similar (generalmente entre 0 y 1), lo que ayuda a la red neuronal a procesar la información de manera eficiente.

5.4. Bucle de Juego

El bucle principal del juego está implementado como un `AnimationTimer` en JavaFX. Gestiona la velocidad de la simulación y controla la lógica de actualización:

```

java

private void startGameLoop() {
    gameLoop = new AnimationTimer() {
        private long lastUpdate = 0;
        private long accumulatedTime = 0;

        @Override
        public void handle(long now) {

```

```

        // (Lógica de control de tiempo)

        // Ejecutar actualizaciones de lógica hasta ponerse al día
        con el tiempo acumulado
        while (accumulatedTime >= timePerFrame && updateCount <
maxUpdates) {
            // Actualizar juego
            game.update(population.getAgents());
            accumulatedTime -= timePerFrame;
            updateCount++;
        }

        // (Actualización de visualizaciones, comprobaciones,
        etc.)
    }

};

gameLoop.start();
}

```

Este diseño permite:

- **Ejecución a velocidad constante:** Las actualizaciones lógicas ocurren a intervalos fijos.
- **Aceleración variable:** La simulación puede ejecutarse más rápido sin afectar la lógica.
- **Control de carga:** Se limita el número máximo de actualizaciones por frame para evitar bloqueos.

6. Características Adicionales

6.1. Control de Velocidad de Simulación

La simulación puede acelerarse hasta 20 veces su velocidad normal, permitiendo un entrenamiento más rápido:

```

java
long timePerFrame = BASE_FRAME_TIME / gameSpeed;

```

6.2. Visualización de la Red Neuronal

La ventana de visualización de la red neuronal muestra en tiempo real las conexiones, pesos y activaciones de la red neuronal del mejor agente, facilitando la comprensión del proceso de toma de decisiones.

6.3. Gráficos de Rendimiento

El sistema registra y muestra gráficamente tres métricas clave:

- **Mejor fitness histórico:** Mejor agente de todas las generaciones
- **Mejor fitness de la generación actual:** Rendimiento del mejor agente actual
- **Fitness promedio de la generación actual:** Indicador del nivel general de la población

6.4. Controles Avanzados

- Opción para mostrar solo el mejor agente o todos los agentes.
- Reinicio automático al extinguirse la población.
- Capacidad para pausar, continuar y avanzar por generaciones.

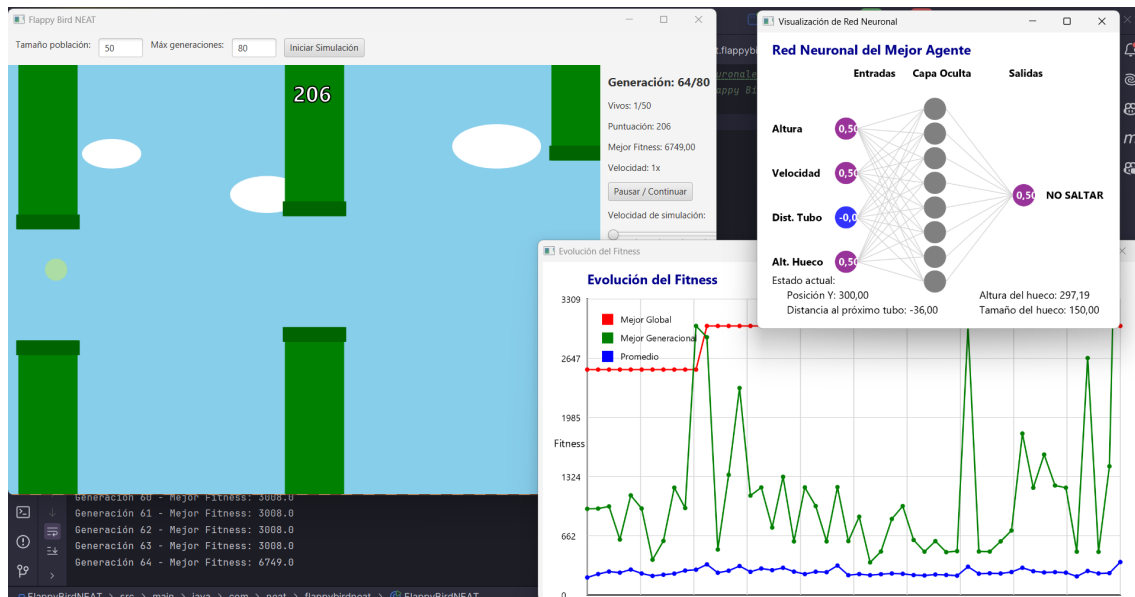
7. Ejecuciones y conclusiones

Esta implementación del algoritmo NEAT para Flappy Bird demuestra cómo las técnicas de neuroevolución pueden aplicarse con éxito para resolver problemas de control en juegos. El sistema logra crear agentes que, a través de múltiples generaciones, aprenden a jugar al juego de forma cada vez más eficiente.

El proyecto incluye una interfaz gráfica completa que permite visualizar el proceso de aprendizaje y analizar el rendimiento de los agentes, lo que lo convierte en una herramienta educativa valiosa para entender los algoritmos evolutivos y las redes neuronales.

El algoritmo no siempre encuentra el mejor individuo, observamos que tiene cierta tendencia a encontrarlo rápido (menos de 30 generaciones) pero lo encuentra en la misma medida en generaciones mas cercanas a la 80. Esto puede suceder por el estancamiento o la pérdida de diversidad genética, en la que todos los individuos acaban codificando una solución en la que el agente solo avanza hacia adelante.

A continuación, se muestra una ejecución que alcanza el individuo óptimo, en el que nunca se detiene la ejecución pues nunca falla el agente.



Observamos que una vez se alcanzan fitness de 2000 o 3000 (20 casillas aproximadamente), se puede afirmar con cierta certeza que el algoritmo encontrara el óptimo.